



当我们在算时间复杂度的时候，我们在做什么？

1. 确定**基本运算**
2. 找出**变化规律**
3. 联系**跳出条件**



题型1 单层循环

【2011】 设 n 是描述问题规模的非负整数，下面程序片段的时间复杂度是

```
x = 2;  
while(x < n/2)  
    x = 2 * x;
```

A. $O(\log_2 n)$

B. $O(n)$

C. $O(n \log_2 n)$

D. $O(n^2)$

1. 确定基本运算
2. 找出变化规律
3. 联系跳出条件



题型1 单层循环

【2019】 设 n 是描述问题规模的非负整数，下列程序段的时间复杂度是

```
x = 0;  
while(n >= (x+1) * (x+1))  
    x = x+1;
```

A. $O(\log_2 n)$

B. $O(n^{1/2})$

C. $O(n)$

D. $O(n^2)$

1. 确定基本运算
2. 找出变化规律
3. 联系跳出条件



题型1 单层循环

【2017】 下列函数的时间复杂度是

```
int func(int n){  
    int i = 0; sum = 0;  
    while(sum < n) sum += ++i;  
    return i; }
```

- 1.确定基本运算
- 2.找出变化规律
- 3.联系跳出条件



题型2 递归

【2012】求整数 n ($n \geq 0$) 阶乘的算法如下，其时间复杂度是

```
int fact(int n){  
    if(n <= 1) return 1;  
    return n * fact(n - 1); }
```

A. $O(\log_2 n)$

B. $O(n)$

C. $O(n \log_2 n)$

D. $O(n^2)$

1. 确定基本运算
2. 找出变化规律
3. 联系跳出条件



题型3 for循环嵌套

层层无关

【2014】 下列程序段的时间复杂度是

```
count = 0;
for(k = 1; k <= n; k *= 2)
    for(j = 1; j <= n; j++)
        count++;
```

A. $O(\log_2 n)$

B. $O(n)$

C. $O(n \log_2 n)$

D. $o(n^2)$

1. 确定基本运算
2. 找出变化规律
3. 联系跳出条件



题型3 for循环嵌套

层层制约

【2022】 下列程序段的时间复杂度是

```
int sum = 0;
for(int i = 1; i < n; i *= 2)
    for(int j = 0; j < i; j++)
        sum++;
```

A. $O(\log_2 n)$

B. $O(n)$

C. $O(n\log_2 n)$

D. $o(n^2)$

1. 确定基本运算
2. 找出变化规律
3. 联系跳出条件



题型3 for循环嵌套

层层制约

【2025】 下列程序段的时间复杂度是

```
int count = 0;
for(int i = 1; i * i <= n; i++)
    for(int j = 1; j <= i; j++)
        count++;
```

A. $O(\log_2 n)$

B. $O(n)$

C. $O(n\log_2 n)$

D. $o(n^2)$

1. 确定基本运算
2. 找出变化规律
3. 联系跳出条件



习题训练

1. 下列算法的时间复杂度为

```
void fun(int n){  
    for(int i = 1; i < n; i *= 2)  
        for(int j = 0; j < n; j++)  
            printf("A");  
  
    for(int i = 0; i < n; i++)  
        for(int j = 0; j < n; j++)  
            printf("B");  
}
```



习题训练

2. 下列算法的时间复杂度为

```
void fun(int n){  
    for(int i = 0; i < n; i++)  
        for(int j = 1; j < n; j *= 2)  
            printf("everything's gonna BOK!");  
}
```



习题训练

3. 下列算法的时间复杂度为

```
int fun(int n){  
    if(n <= 1)  
        return 1;  
    else  
        return fun(n - 1) + 1;  
}
```



习题训练

4. 下列算法的时间复杂度为

```
int count = 1;
for(int i = 1; i <= n; i++)
{
    for(int j = 2*i; j <= n; j++)
        count = count + j;
}
```



祝你今天特别特别开心^•u-^